Severn Edge AI — Complete Implementation Specification

> **Purpose:** This document is the single source of truth for implementing Severn Edge AI. It is designed as input for an AI coding agent to generate production-quality code.

Table of Contents

1. Project Overview
2. Current Architecture
3. Problems to Solve
4. Implementation Roadmap
5. M1: Classroom Stability
6. M2: Learning Outcomes
7. M3: PBL + Showcase
8. Technical Reference
9. Testing Requirements

---

1. Project Overview

What Is This?

Severn Edge AI is a **machine learning education platform** for 5th graders (ages 10-11). Students:

1. Connect to an Arduino via Bluetooth
2. Record gesture samples (wave, shake, custom motions)
3. Train a neural network in-browser using TensorFlow.js
4. Deploy the model to the Arduino over Bluetooth
5. Test real-time inference on the physical device

Key insight: Students see the actual sensor numbers, not magic. They learn that AI is pattern matching on data.

Technical Stack

Layer	Technology
Frontend	React 18 + TypeScript + Vite + Tailwind CSS

Layer	Technology
ML Training	TensorFlow.js (in-browser)
Inference Engine	SimpleNN (custom C++, not TFLite)
Communication	Web Bluetooth API
Hardware	Arduino Nano 33 BLE Sense (Rev1/Rev2)
Persistence	IndexedDB (to be implemented)
Hosting	GitHub Pages (static, no backend)

Repository Structure

```
severn-edge-ai/
├── firmware/                  # Arduino C++ (PlatformIO)
│   ├── src/
│   │   ├── main.cpp             # BLE setup, main loop
│   │   ├── config.h             # Constants (MUST match web app)
│   │   ├── simple_nn.cpp/h      # Neural network inference
│   │   ├── inference.cpp/h      # Sliding window, normalization
│   │   ├── flash_storage.cpp/h  # Model persistence
│   │   ├── sensor_bmi270.cpp    # Rev2 IMU driver
│   │   └── sensor_lsm9ds1.cpp   # Rev1 IMU driver
│   └── platformio.ini
│
├── web-app/                   # React application
│   ├── src/
│   │   ├── App.tsx              # Wizard flow state machine
│   │   ├── pages/
│   │   │   ├── ConnectPage.tsx
│   │   │   ├── CollectPage.tsx  # Two-phase: setup → recording
│   │   │   ├── TrainPage.tsx
│   │   │   └── TestPage.tsx
│   │   ├── services/
│   │   │   ├── bleService.ts    # BLE connection handling
│   │   │   ├── bleParser.ts     # Binary packet parsing
│   │   │   ├── trainingService.ts  # TensorFlow.js training
│   │   │   ├── modelExportService.ts
│   │   │   └── bleModelUploadService.ts
│   │   ├── config/
│   │   │   └── constants.ts     # Shared constants (MUST match firmware)
```

```
│   │   ├── components/
│   │   └── types/
│   │       └── index.ts
│   └── package.json
│
└── docs/
    ├── CLASSROOM_GUIDE.md
    └── AI_CONTEXT.md
```

---

2. Current Architecture

User Flow (What Exists Today)

```
ConnectPage → CollectPage → TrainPage → TestPage
    │           │           │           │
    │           │           │           │
 Bluetooth    Record 10+    TF.js train   Live inference
 pairing      samples per   + BLE upload  on Arduino
              gesture
```

BLE Communication

Service & Characteristics

```
Service UUID: 19b10001-e8f2-537e-4f6c-d104768a1214

Characteristics:
├── Sensor (19b10002-...): NOTIFY, streams sensor data at 25Hz
├── Control (19b10003-...): WRITE, commands like "start", "stop"
└── Model (19b10004-...): WRITE, receives model weight chunks
```

Sensor Packet Format (17 bytes)

```
┌──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┐
│ ax   │ ay   │ az   │ gx   │ gy   │ gz   │ seq  │ ts   │ crc  │
├──────┼──────┼──────┼──────┼──────┼──────┼──────┼──────┼──────┤
│2 B   │2 B   │2 B   │2 B   │2 B   │2 B   │2 B   │2 B   │1 B   │
└──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┘

 int16 (accelerometer)   int16 (gyroscope)   uint16  uint8
```

Critical Constants (Must Match)

Parameter	Web App (`constants.ts`)	Firmware (`config.h`)
WINDOW_SIZE	100	100
NUM_AXES	6	6
NN_HIDDEN_SIZE	32	32
NN_INPUT_SIZE	600	600
NN_MAX_CLASSES	8	8
NORM_ACCEL	4.0	4.0f
NORM_GYRO	500.0	500.0f

Neural Network Architecture

```
Input: 600 floats (100 timesteps × 6 axes)
   ↓
Dense(32, ReLU)
   ↓
Dense(N, Softmax)  // N = number of gesture classes
   ↓
Output: Gesture probabilities
```

What's Working

- ✅ BLE connection and sensor streaming
- ✅ Sample recording with quality validation
- ✅ TensorFlow.js training in browser
- ✅ SimpleNN model export
- ✅ OTA model upload via BLE
- ✅ Real-time inference on Arduino
- ✅ "See what the AI sees" sensor panel

What's Broken/Missing

- ❌ No persistence — refresh loses everything

- ❌ Disconnects fail silently
- ❌ No knowledge checks — students rush through
- ❌ No project ownership — feels like tutorial
- ❌ No shareable artifact — nothing to show parents

---

3. Problems to Solve

Classroom Pilot Observations

Problem 1: Silent Disconnects

Symptom: Arduino disconnects, student confused, teacher spends 5 min debugging. **Root cause:** No reconnect logic. No visual feedback. Characteristics not rehydrated. **Solution:** Connection state machine with auto-reconnect and clear UI.

Problem 2: Refresh Wipes Everything

Symptom: Chromebook refreshes, 30 minutes of work gone. **Root cause:** All state in memory, no persistence. **Solution:** IndexedDB with auto-save and session recovery.

Problem 3: Students Rush Through

Symptom: Kids click through without learning, collect garbage data. **Root cause:** No gates, no understanding checks. **Solution:** 2 knowledge check gates at critical transitions.

Problem 4: No Ownership

Symptom: Feels like following instructions, not "my project." **Root cause:** No project framing, no reflection, no artifact. **Solution:** Project Brief, Design Journal, Portfolio with download.

Problem 5: Nothing to Show Parents

Symptom: Student goes home, can't demonstrate what they built. **Root cause:** App on school Chromebook, IndexedDB doesn't travel. **Solution:** Self-contained HTML portfolio download.

---

4. Implementation Roadmap

Milestones

Milestone	Focus	Target
M1	Classroom Stability	Before next pilot
M2	Learning Outcomes	2 weeks after M1
M3	PBL + Showcase	4 weeks after M1

Dependency Graph

```
M1: Stability
├── 1.1 Connection State Machine
│   └── 1.2 Status Pill + Reconnect Modal
│       └── 6.1a BLE Upload Retry
├── 2.1 IndexedDB Persistence
│   └── 2.2 Session Recovery UI
└── 7.1 Documentation

M2: Learning
├── 3.1 Knowledge Check Gates (depends on 2.1)
├── 3.2 Data Quality Feedback
├── 3.3 "Idle" Class Explanation
├── 4.3 Edge AI Facts Panel
└── 4.4 Measured Inference Timing (optional)

M3: Showcase
├── 4.1 Project Brief (depends on 2.1)
│   └── 4.2 Design Journal
├── 5.1 Badge System (depends on 2.1)
├── 5.2 Portfolio Page (depends on 4.1, 4.2, 5.1)
└── 5.3 HTML Portfolio Download (depends on 5.2)
```

---

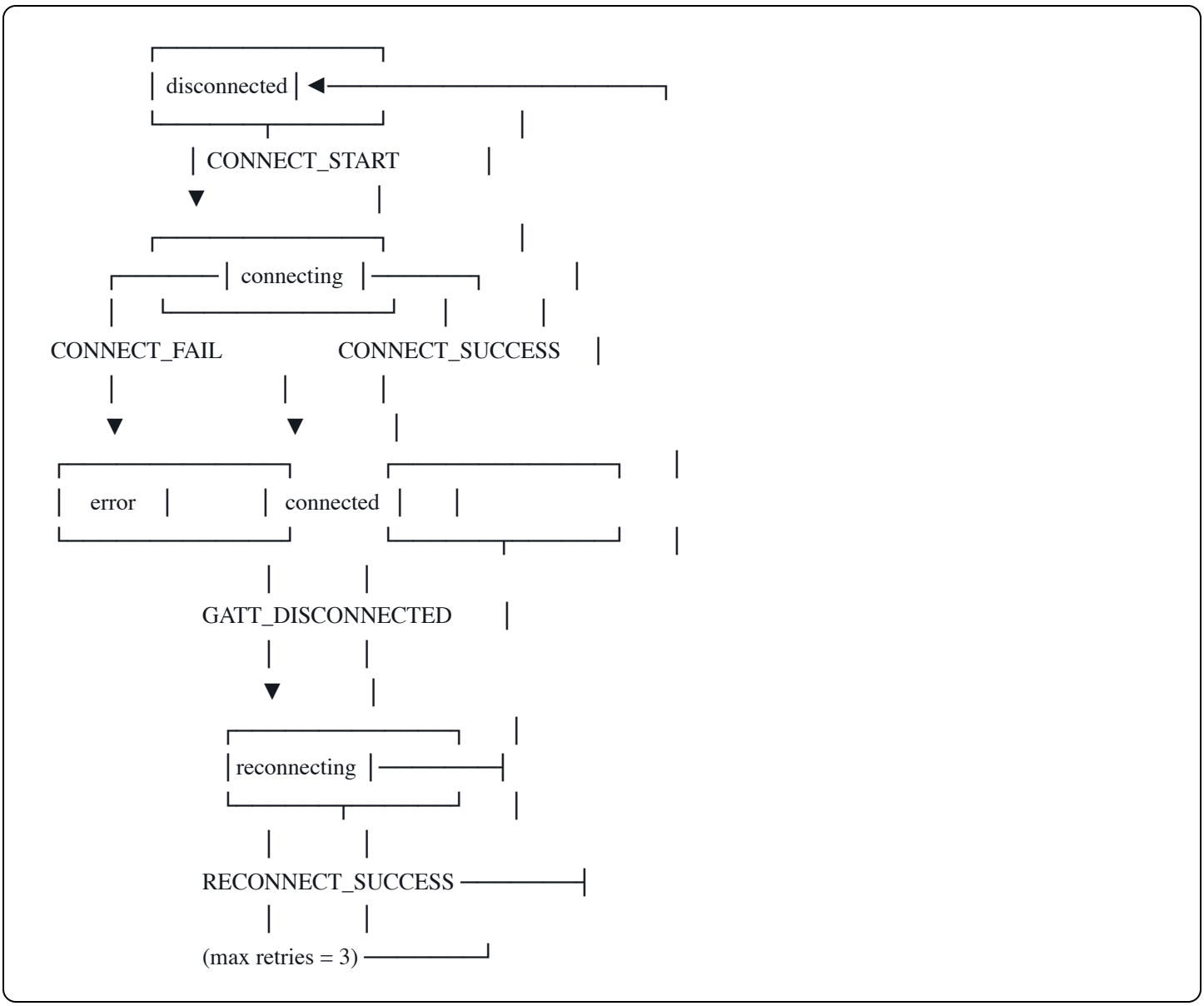
5. M1: Classroom Stability

5.1 Connection State Machine

States

```typescript
type ConnectionState =
  | 'disconnected'   // No device
  | 'connecting'     // requestDevice() in progress
  | 'connected'      // Full BLE pipeline ready
  | 'reconnecting'   // Auto-retry in progress
  | 'error';         // Failed, needs user action
```

State Transitions



Implementation

File: `state/connectionStore.ts`

```typescript
```

```typescript
import { create } from 'zustand';

type ConnectionState = 'disconnected' | 'connecting' | 'connected' | 'reconnecting' | 'error';

interface ConnectionStore {
  state: ConnectionState;
  deviceName: string | null;
  error: string | null;
  reconnectAttempt: number;

  // Actions
  startConnecting: () => void;
  connectSuccess: (deviceName: string) => void;
  connectFail: (error: string) => void;
  disconnect: () => void;
  startReconnecting: () => void;
  reconnectSuccess: () => void;
  reconnectFail: () => void;
  reset: () => void;
}

export const useConnectionStore = create<ConnectionStore>((set, get) => ({
  state: 'disconnected',
  deviceName: null,
  error: null,
  reconnectAttempt: 0,

  startConnecting: () => set({ state: 'connecting', error: null }),

  connectSuccess: (deviceName) => {
    localStorage.setItem('lastDeviceName', deviceName);
    set({ state: 'connected', deviceName, error: null, reconnectAttempt: 0 });
  },

  connectFail: (error) => set({ state: 'error', error }),

  disconnect: () => set({ state: 'disconnected', error: null }),

  startReconnecting: () => set((s) => ({
    state: 'reconnecting',
    reconnectAttempt: s.reconnectAttempt + 1,
  })),
```

```
  reconnectSuccess: () => set({ state: 'connected', reconnectAttempt: 0 }),

  reconnectFail: () => {
    const { reconnectAttempt } = get();
    if (reconnectAttempt >= 3) {
      set({ state: 'disconnected', reconnectAttempt: 0 });
    }
  },

  reset: () => set({
    state: 'disconnected',
    deviceName: null,
    error: null,
    reconnectAttempt: 0,
  }),
}));
```

File: **services/bleService.ts** (key methods)

```typescript
```

```
import { useConnectionStore } from '../state/connectionStore';

const SERVICE_UUID = '19b10001-e8f2-537e-4f6c-d104768a1214';
const SENSOR_CHAR_UUID = '19b10002-e8f2-537e-4f6c-d104768a1214';
const CONTROL_CHAR_UUID = '19b10003-e8f2-537e-4f6c-d104768a1214';
const MODEL_CHAR_UUID = '19b10004-e8f2-537e-4f6c-d104768a1214';

class BLEService {
  private device: BluetoothDevice | null = null;
  private gattServer: BluetoothRemoteGATTServer | null = null;
  private sensorChar: BluetoothRemoteGATTCharacteristic | null = null;
  private controlChar: BluetoothRemoteGATTCharacteristic | null = null;
  private modelChar: BluetoothRemoteGATTCharacteristic | null = null;

  private store = useConnectionStore.getState();

  async connect(): Promise<void> {
    this.store.startConnecting();

    try {
      this.device = await navigator.bluetooth.requestDevice({
        filters: [{ namePrefix: 'SevernEdgeAI-' }],
        optionalServices: [SERVICE_UUID],
      });

      this.device.addEventListener('gattserverdisconnected', this.handleDisconnect);

      await this.connectFull();

      this.store.connectSuccess(this.device.name || 'Unknown');
    } catch (error) {
      this.store.connectFail(error instanceof Error ? error.message : 'Connection failed');
      throw error;
    }
  }

  /**
   * Full BLE rehydration — MUST be called on initial connect AND reconnect.
   * Just calling gatt.connect() is NOT enough.
   */
  private async connectFull(): Promise<void> {
    if (!this.device?.gatt) throw new Error('No device');
```

```
  // Step 1: Connect to GATT server
  this.gattServer = await this.device.gatt.connect();

  // Step 2: Get the service
  const service = await this.gattServer.getPrimaryService(SERVICE_UUID);

  // Step 3: Get all characteristics
  this.sensorChar = await service.getCharacteristic(SENSOR_CHAR_UUID);
  this.controlChar = await service.getCharacteristic(CONTROL_CHAR_UUID);
  this.modelChar = await service.getCharacteristic(MODEL_CHAR_UUID);

  // Step 4: Subscribe to notifications
  await this.sensorChar.startNotifications();
  this.sensorChar.addEventListener('characteristicvaluechanged', this.handleSensorData);
}

private handleDisconnect = async (): Promise<void> => {
  // Don't auto-reconnect if user intentionally disconnected
  if (this.store.state === 'disconnected') return;

  this.store.startReconnecting();

  const MAX_RETRIES = 3;
  const BACKOFF_MS = [2000, 4000, 8000];

  for (let attempt = 0; attempt < MAX_RETRIES; attempt++) {
    try {
      await this.delay(BACKOFF_MS[attempt]);
      await this.connectFull();
      this.store.reconnectSuccess();
      return;
    } catch (error) {
      console.warn(`Reconnect attempt ${attempt + 1} failed:`, error);
      this.store.reconnectFail();
    }
  }

  // All retries exhausted
  this.store.disconnect();
};

private handleSensorData = (event: Event): void => {
  const characteristic = event.target as BluetoothRemoteGATTCharacteristic;
  const value = characteristic.value;
```

```
    if (!value) return;

    // Parse 17-byte packet
    const data = new DataView(value.buffer);
    const packet = {
      ax: data.getInt16(0, true) / 8192.0,  // Convert to g
      ay: data.getInt16(2, true) / 8192.0,
      az: data.getInt16(4, true) / 8192.0,
      gx: data.getInt16(6, true) / 16.4,    // Convert to dps
      gy: data.getInt16(8, true) / 16.4,
      gz: data.getInt16(10, true) / 16.4,
      seq: data.getUint16(12, true),
      ts: data.getUint16(14, true),
      crc: data.getUint8(16),
    };

    // Emit to listeners
    this.emit('sensorData', packet);
  };

  private delay(ms: number): Promise<void> {
    return new Promise(resolve => setTimeout(resolve, ms));
  }

  // ... event emitter implementation ...
}

export const bleService = new BLEService();
```

UI Components

File: `components/ConnectionStatusPill.tsx`

```typescript
```

```
import { useConnectionStore } from '../state/connectionStore';

export function ConnectionStatusPill() {
  const { state, deviceName } = useConnectionStore();

  const config = {
    disconnected: { color: 'bg-red-500', text: 'Disconnected', pulse: false },
    connecting: { color: 'bg-yellow-500', text: 'Connecting...', pulse: true },
    connected: { color: 'bg-green-500', text: deviceName || 'Connected', pulse: false },
    reconnecting: { color: 'bg-yellow-500', text: 'Reconnecting...', pulse: true },
    error: { color: 'bg-red-500', text: 'Error', pulse: false },
  }[state];

  return (
    <div className="flex items-center gap-2 px-3 py-1.5 bg-gray-100 rounded-full">
      <div className={`w-2.5 h-2.5 rounded-full ${config.color} ${config.pulse ? 'animate-pulse' : ''}`} />
      <span className="text-sm font-medium text-gray-700">{config.text}</span>
    </div>
  );
}
```

File: `components/ReconnectModal.tsx`

```typescript
```

```
import { useConnectionStore } from '../state/connectionStore';
import { bleService } from '../services/bleService';

export function ReconnectModal() {
  const { state, reconnectAttempt, deviceName } = useConnectionStore();

  // Only show when reconnecting or disconnected (not on ConnectPage)
  const isVisible = state === 'reconnecting' || state === 'disconnected';
  const isReconnecting = state === 'reconnecting';

  if (!isVisible) return null;

  const handleChooseDevice = async () => {
    await bleService.connect();
  };

  return (
    <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50">
      <div className="bg-white rounded-xl p-6 max-w-sm mx-4 text-center">
        {isReconnecting ? (
          <>
            <div className="w-12 h-12 border-4 border-blue-500 border-t-transparent rounded-full animate-spin mx-auto mb-4"
            <h2 className="text-xl font-semibold mb-2">Connection Lost</h2>
            <p className="text-gray-600 mb-4">
              Trying to reconnect... (Attempt {reconnectAttempt}/3)
            </p>
          </>
        ) : (
          <>
            <div className="text-4xl mb-4">😕 </div>
            <h2 className="text-xl font-semibold mb-2">Couldn't Reconnect</h2>
            <p className="text-gray-600 mb-4">
              Make sure your Arduino is powered on and nearby.
              {deviceName && ` Look for "${deviceName}".`}
            </p>
            <button
              onClick={handleChooseDevice}
              className="w-full py-3 bg-blue-600 text-white rounded-lg font-medium hover:bg-blue-700"
            >
              Choose Device
            </button>
          </>
        )}
```

```
      </div>
    </div>
  );
}
```

5.2 IndexedDB Persistence

Schema

```typescript
```

```typescript
// File: storage/schema.ts

export interface StudentSession {
  id: string;                    // UUID
  createdAt: Date;
  lastUpdated: Date;

  // Project-Based Learning
  projectBrief: ProjectBrief | null;
  designJournal: DesignJournalEntry[];

  // Training Data (CRITICAL - most valuable)
  gestures: GestureLabel[];
  samples: Sample[];             // 600 floats × 30+ samples

  // Progress
  badges: BadgeId[];
  checkpointsPassed: CheckpointId[];
  currentStage: AppStage;
  trainingAccuracy: number | null;

  // Connection
  lastDeviceName: string | null;
}

export interface ProjectBrief {
  studentName: string;
  name: string;                  // Project name
  problemStatement: string;
  useCase: 'accessibility' | 'gaming' | 'art' | 'communication' | 'other';
  gestureIdeas?: string;
}

export interface DesignJournalEntry {
  id: string;
  prompt: 'after-preview' | 'after-train' | 'after-test';
  response: string;
  timestamp: Date;
}

export interface GestureLabel {
  id: string;
  name: string;
```

```
  emoji?: string;
}

export interface Sample {
  id: string;
  gestureId: string;
  data: number[];                    // 600 floats (100 × 6)
  timestamp: Date;
}

export type BadgeId =
  | 'connected'
  | 'sensor-explorer'
  | 'designer'
  | 'data-scientist'
  | 'ai-trainer'
  | 'edge-engineer'
  | 'sharp-shooter';

export type CheckpointId = 'gate-1-sensor' | 'gate-2-gesture';

export type AppStage =
  | 'connect'
  | 'project-brief'
  | 'preview'
  | 'collect'
  | 'train'
  | 'test'
  | 'portfolio';
```

IndexedDB Wrapper

```typescript
```

```typescript
// File: storage/db.ts

import { openDB, DBSchema, IDBPDatabase } from 'idb';
import { StudentSession } from './schema';

interface SevernEdgeDB extends DBSchema {
  sessions: {
    key: string;
    value: StudentSession;
    indexes: {
      'by-updated': Date;
    };
  };
}

const DB_NAME = 'severn-edge-ai';
const DB_VERSION = 1;

let dbPromise: Promise<IDBPDatabase<SevernEdgeDB>> | null = null;

async function getDb(): Promise<IDBPDatabase<SevernEdgeDB>> {
  if (!dbPromise) {
    dbPromise = openDB<SevernEdgeDB>(DB_NAME, DB_VERSION, {
      upgrade(db) {
        const store = db.createObjectStore('sessions', { keyPath: 'id' });
        store.createIndex('by-updated', 'lastUpdated');
      },
    });
  }
  return dbPromise;
}

export async function getSession(id: string): Promise<StudentSession | undefined> {
  const db = await getDb();
  return db.get('sessions', id);
}

export async function saveSession(session: StudentSession): Promise<void> {
  const db = await getDb();
  session.lastUpdated = new Date();
  await db.put('sessions', session);
}
```

```typescript
export async function getMostRecentSession(): Promise<StudentSession | undefined> {
  const db = await getDb();
  const all = await db.getAllFromIndex('sessions', 'by-updated');
  return all[all.length - 1];
}

export async function deleteSession(id: string): Promise<void> {
  const db = await getDb();
  await db.delete('sessions', id);
}

export async function clearAllSessions(): Promise<void> {
  const db = await getDb();
  await db.clear('sessions');
}

export function createEmptySession(): StudentSession {
  return {
    id: crypto.randomUUID(),
    createdAt: new Date(),
    lastUpdated: new Date(),
    projectBrief: null,
    designJournal: [],
    gestures: [],
    samples: [],
    badges: [],
    checkpointsPassed: [],
    currentStage: 'connect',
    trainingAccuracy: null,
    lastDeviceName: null,
  };
}
```

Session Store (Zustand + IndexedDB)

```typescript
```

```typescript
// File: state/sessionStore.ts

import { create } from 'zustand';
import { StudentSession, Sample, GestureLabel, BadgeId } from '../storage/schema';
import { saveSession, getMostRecentSession, createEmptySession } from '../storage/db';
import { debounce } from '../utils/debounce';

interface SessionStore {
  session: StudentSession | null;
  isLoading: boolean;
  hasRecoverySession: boolean;

  // Initialization
  initialize: () => Promise<void>;
  recoverSession: (session: StudentSession) => void;
  startFresh: () => void;

  // Updates (all trigger auto-save)
  setProjectBrief: (brief: StudentSession['projectBrief']) => void;
  setGestures: (gestures: GestureLabel[]) => void;
  addSample: (sample: Sample) => void;
  removeSample: (sampleId: string) => void;
  addBadge: (badge: BadgeId) => void;
  passCheckpoint: (checkpoint: CheckpointId) => void;
  setStage: (stage: AppStage) => void;
  setTrainingAccuracy: (accuracy: number) => void;
  addJournalEntry: (entry: DesignJournalEntry) => void;
  setDeviceName: (name: string) => void;
}

// Debounced save to avoid hammering IndexedDB
const debouncedSave = debounce(async (session: StudentSession) => {
  await saveSession(session);
}, 1000);

export const useSessionStore = create<SessionStore>((set, get) => ({
  session: null,
  isLoading: true,
  hasRecoverySession: false,

  initialize: async () => {
    const existing = await getMostRecentSession();
    if (existing && (existing.samples.length > 0 || existing.projectBrief)) {
```

```
    set({ hasRecoverySession: true, isLoading: false });
  } else {
    set({ session: createEmptySession(), isLoading: false });
  }
},

recoverSession: (session) => {
  set({ session, hasRecoverySession: false });
},

startFresh: () => {
  set({ session: createEmptySession(), hasRecoverySession: false });
},

setProjectBrief: (brief) => {
  const { session } = get();
  if (!session) return;
  const updated = { ...session, projectBrief: brief };
  set({ session: updated });
  debouncedSave(updated);
},

setGestures: (gestures) => {
  const { session } = get();
  if (!session) return;
  const updated = { ...session, gestures };
  set({ session: updated });
  debouncedSave(updated);
},

addSample: (sample) => {
  const { session } = get();
  if (!session) return;
  const updated = { ...session, samples: [...session.samples, sample] };
  set({ session: updated });
  // Immediate save for samples (most valuable data)
  saveSession(updated);
},

removeSample: (sampleId) => {
  const { session } = get();
  if (!session) return;
  const updated = {
    ...session,
```

```
    samples: session.samples.filter((s) => s.id !== sampleId),
  };
  set({ session: updated });
  debouncedSave(updated);
},

addBadge: (badge) => {
  const { session } = get();
  if (!session || session.badges.includes(badge)) return;
  const updated = { ...session, badges: [...session.badges, badge] };
  set({ session: updated });
  saveSession(updated); // Immediate save
},

passCheckpoint: (checkpoint) => {
  const { session } = get();
  if (!session || session.checkpointsPassed.includes(checkpoint)) return;
  const updated = {
    ...session,
    checkpointsPassed: [...session.checkpointsPassed, checkpoint],
  };
  set({ session: updated });
  saveSession(updated);
},

setStage: (stage) => {
  const { session } = get();
  if (!session) return;
  const updated = { ...session, currentStage: stage };
  set({ session: updated });
  debouncedSave(updated);
},

setTrainingAccuracy: (accuracy) => {
  const { session } = get();
  if (!session) return;
  const updated = { ...session, trainingAccuracy: accuracy };
  set({ session: updated });
  saveSession(updated);
},

addJournalEntry: (entry) => {
  const { session } = get();
  if (!session) return;
```

```
    const updated = { ...session, designJournal: [...session.designJournal, entry] };
    set({ session: updated });
    debouncedSave(updated);
  },

  setDeviceName: (name) => {
    const { session } = get();
    if (!session) return;
    const updated = { ...session, lastDeviceName: name };
    set({ session: updated });
    debouncedSave(updated);
  },
}));
```

Session Recovery UI

```typescript
```

```
// File: components/SessionRecoveryBanner.tsx

import { useSessionStore } from '../state/sessionStore';
import { getMostRecentSession } from '../storage/db';
import { useEffect, useState } from 'react';
import { StudentSession } from '../storage/schema';

export function SessionRecoveryBanner() {
  const { hasRecoverySession, recoverSession, startFresh } = useSessionStore();
  const [recoveryData, setRecoveryData] = useState<StudentSession | null>(null);

  useEffect(() => {
    if (hasRecoverySession) {
      getMostRecentSession().then(setRecoveryData);
    }
  }, [hasRecoverySession]);

  if (!hasRecoverySession || !recoveryData) return null;

  const sampleCount = recoveryData.samples.length;
  const gestureCount = recoveryData.gestures.length;
  const timeAgo = formatRelativeTime(recoveryData.lastUpdated);

  return (
    <div className="bg-blue-50 border-l-4 border-blue-500 p-4 mb-4">
      <div className="flex items-start gap-3">
        <span className="text-2xl">📂 </span>
        <div className="flex-1">
          <h3 className="font-semibold text-blue-900">Found Your Previous Work!</h3>
          <p className="text-blue-700 text-sm mt-1">
            {gestureCount} gestures, {sampleCount} samples recorded
            <br />
            Last updated {timeAgo}
          </p>
          <div className="flex gap-3 mt-3">
            <button
              onClick={() => recoverSession(recoveryData)}
              className="px-4 py-2 bg-blue-600 text-white rounded-lg font-medium hover:bg-blue-700"
            >
              Continue
            </button>
            <button
              onClick={startFresh}
```

```
            className="px-4 py-2 text-blue-600 hover:underline"
          >
            Start Fresh
          </button>
        </div>
      </div>
    </div>
  </div>
  );
}

function formatRelativeTime(date: Date): string {
  const seconds = Math.floor((Date.now() - date.getTime()) / 1000);
  if (seconds < 60) return 'just now';
  if (seconds < 3600) return `${Math.floor(seconds / 60)} minutes ago`;
  if (seconds < 86400) return `${Math.floor(seconds / 3600)} hours ago`;
  return `${Math.floor(seconds / 86400)} days ago`;
}
```

5.3 BLE Upload Retry

```typescript
```

```typescript
// File: services/bleModelUploadService.ts (key improvements)

export async function uploadModel(
  modelChar: BluetoothRemoteGATTCharacteristic,
  modelData: Uint8Array,
  onProgress: (percent: number) => void
): Promise<void> {
  const CHUNK_SIZE = 240;
  const MAX_RETRIES = 3;
  const RETRY_DELAYS = [500, 1000, 2000];

  const totalChunks = Math.ceil(modelData.length / CHUNK_SIZE);

  for (let i = 0; i < totalChunks; i++) {
    const offset = i * CHUNK_SIZE;
    const chunk = modelData.slice(offset, offset + CHUNK_SIZE);

    let success = false;

    for (let retry = 0; retry < MAX_RETRIES && !success; retry++) {
      try {
        await writeChunk(modelChar, offset, chunk);
        success = true;
      } catch (error) {
        console.warn(`Chunk ${i} failed (attempt ${retry + 1}):`, error);
        if (retry < MAX_RETRIES - 1) {
          await delay(RETRY_DELAYS[retry]);
        }
      }
    }

    if (!success) {
      throw new Error(`Upload failed at ${Math.round((offset / modelData.length) * 100)}%`);
    }

    onProgress(Math.round(((i + 1) / totalChunks) * 100));
  }
}

async function writeChunk(
  char: BluetoothRemoteGATTCharacteristic,
  offset: number,
  data: Uint8Array
```

```
): Promise<void> {
  // Packet format: [offset (4 bytes)] [data (N bytes)]
  const packet = new Uint8Array(4 + data.length);
  const view = new DataView(packet.buffer);
  view.setUint32(0, offset, true);
  packet.set(data, 4);

  await char.writeValue(packet);
}

function delay(ms: number): Promise<void> {
  return new Promise((resolve) => setTimeout(resolve, ms));
}
```

---

6. M2: Learning Outcomes

6.1 Knowledge Check Gates

Gate Definitions

```typescript
```

```typescript
// File: data/knowledgeChecks.ts

export interface KnowledgeCheck {
  id: CheckpointId;
  question: string;
  options: { text: string; correct: boolean }[];
  explanation: string;
  badge?: BadgeId;
}

export const KNOWLEDGE_CHECKS: Record<CheckpointId, KnowledgeCheck> = {
  'gate-1-sensor': {
    id: 'gate-1-sensor',
    question: 'Your AI uses 6 numbers from two sensors. Which sensor measures how fast you SPIN the board?',
    options: [
      { text: 'Accelerometer', correct: false },
      { text: 'Gyroscope', correct: true },
      { text: 'Thermometer', correct: false },
    ],
    explanation:
      'The gyroscope measures rotation speed. When you spin the board, the blue bars (gx, gy, gz) spike!',
    badge: 'sensor-explorer',
  },
  'gate-2-gesture': {
    id: 'gate-2-gesture',
    question:
      'Why is it important that your gestures move DIFFERENT bars on the sensor panel?',
    options: [
      { text: 'So the AI can tell them apart', correct: true },
      { text: "So the Arduino doesn't overheat", correct: false },
      { text: 'So the colors look pretty', correct: false },
    ],
    explanation:
      "If two gestures make the same bars move the same way, the AI sees the same numbers and can't tell them apart!",
    badge: 'designer',
  },
};
```

Knowledge Check Modal

```typescript
```

```
// File: components/KnowledgeCheckModal.tsx

import { useState } from 'react';
import { KnowledgeCheck } from '../data/knowledgeChecks';
import { useSessionStore } from '../state/sessionStore';

interface Props {
  check: KnowledgeCheck;
  onPass: () => void;
}

export function KnowledgeCheckModal({ check, onPass }: Props) {
  const [selected, setSelected] = useState<number | null>(null);
  const [showResult, setShowResult] = useState(false);
  const [attempts, setAttempts] = useState(0);
  const [showUnlock, setShowUnlock] = useState(false);

  const { addBadge, passCheckpoint } = useSessionStore();

  // Teacher unlock: Ctrl+Shift+U
  useEffect(() => {
    const handler = (e: KeyboardEvent) => {
      if (e.ctrlKey && e.shiftKey && e.key === 'U') {
        setShowUnlock(true);
        setTimeout(() => setShowUnlock(false), 10000); // Hide after 10s
      }
    };
    window.addEventListener('keydown', handler);
    return () => window.removeEventListener('keydown', handler);
  }, []);

  const handleSubmit = () => {
    if (selected === null) return;

    const isCorrect = check.options[selected].correct;
    setShowResult(true);

    if (isCorrect) {
      passCheckpoint(check.id);
      if (check.badge) addBadge(check.badge);
    } else {
      setAttempts((a) => a + 1);
    }
```

```
};

const handleContinue = () => {
  if (check.options[selected!].correct) {
    onPass();
  } else {
    setShowResult(false);
    setSelected(null);
  }
};

const handleUnlock = () => {
  passCheckpoint(check.id);
  // Don't award badge on unlock
  onPass();
};

const isCorrect = selected !== null && check.options[selected].correct;

return (
  <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50 p-4">
    <div className="bg-white rounded-xl p-6 max-w-md w-full">
      <h2 className="text-xl font-bold mb-4">🧠 Quick Check</h2>

      <p className="text-gray-700 mb-6">{check.question}</p>

      <div className="space-y-3 mb-6">
        {check.options.map((option, i) => (
          <button
            key={i}
            onClick={() => !showResult && setSelected(i)}
            disabled={showResult}
            className={`w-full p-4 rounded-lg text-left border-2 transition ${
              showResult
                ? option.correct
                  ? 'border-green-500 bg-green-50'
                  : selected === i
                  ? 'border-red-500 bg-red-50 animate-shake'
                  : 'border-gray-200'
                : selected === i
                ? 'border-blue-500 bg-blue-50'
                : 'border-gray-200 hover:border-gray-300'
            }`}
          >
```

```
      {option.text}
    </button>
  ))}
</div>

{showResult && (
  <div
    className={`p-4 rounded-lg mb-4 ${
      isCorrect ? 'bg-green-100 text-green-800' : 'bg-yellow-100 text-yellow-800'
    }`}
  >
    <p className="font-medium mb-1">{isCorrect ? '✅ Correct!' : '❌ Not quite...'}</p>
    <p className="text-sm">{check.explanation}</p>
  </div>
)}

{attempts >= 2 && !showResult && (
  <p className="text-sm text-gray-500 mb-4">
    💡 Hint: Check your notes or ask a neighbor!
  </p>
)}

{!showResult ? (
  <button
    onClick={handleSubmit}
    disabled={selected === null}
    className="w-full py-3 bg-blue-600 text-white rounded-lg font-medium disabled:opacity-50"
  >
    Check Answer
  </button>
) : (
  <button
    onClick={handleContinue}
    className="w-full py-3 bg-blue-600 text-white rounded-lg font-medium"
  >
    {isCorrect ? 'Continue →' : 'Try Again'}
  </button>
)}

{showUnlock && (
  <button
    onClick={handleUnlock}
    className="w-full mt-3 py-2 text-gray-500 text-sm hover:underline"
  >
```

```
          🔓 Unlock for Student
        </button>
      )}
    </div>
  </div>
 );
}
```

6.2 "Idle" Class Explanation

```typescript
// File: components/IdleClassBanner.tsx

interface Props {
 gestureCount: number;
}

export function IdleClassBanner({ gestureCount }: Props) {
 if (gestureCount !== 1) return null;

 return (
  <div className="bg-yellow-50 border-l-4 border-yellow-400 p-4 mb-4">
   <div className="flex items-start gap-2">
    <span>💡 </span>
    <div>
     <p className="font-medium text-yellow-800">Note: We added an "Idle" class automatically.</p>
     <p className="text-sm text-yellow-700 mt-1">
      This helps the AI learn when you're NOT doing your gesture.
     </p>
    </div>
   </div>
  </div>
 );
}
```

6.3 Edge AI Facts Panel

```typescript
```

```
// File: components/EdgeAIFactsPanel.tsx

import { useState } from 'react';

interface Props {
  inferenceTimeMs?: number; // From firmware, if available
}

export function EdgeAIFactsPanel({ inferenceTimeMs }: Props) {
  const [isOpen, setIsOpen] = useState(false);

  return (
    <div className="border rounded-xl overflow-hidden">
      <button
        onClick={() => setIsOpen(!isOpen)}
        className="w-full p-4 flex items-center justify-between bg-blue-50 hover:bg-blue-100"
      >
        <span className="font-medium text-blue-900">🧠 Did you know? You're doing Edge AI!</span>
        <span className="text-blue-600">{isOpen ? '▲' : '▼'}</span>
      </button>

      {isOpen && (
        <div className="p-4 bg-white">
          <p className="text-gray-700 mb-4">
            Your AI runs entirely on this tiny chip — no internet needed.
          </p>

          {inferenceTimeMs && (
            <p className="text-lg font-semibold text-green-600 mb-4">
              ⚡ Your Arduino predicted in {inferenceTimeMs.toFixed(1)}ms
            </p>
          )}

          <table className="w-full text-sm border-collapse">
            <thead>
              <tr className="border-b">
                <th className="text-left py-2 text-gray-500">Cloud AI</th>
                <th className="text-left py-2 text-blue-600">Edge AI (you!)</th>
              </tr>
            </thead>
            <tbody className="text-gray-700">
              <tr className="border-b">
                <td className="py-2">Sends data far away</td>
```

```
            <td className="py-2">Runs on the device</td>
          </tr>
          <tr className="border-b">
            <td className="py-2">Needs internet</td>
            <td className="py-2">Works offline</td>
          </tr>
          <tr className="border-b">
            <td className="py-2">Can be slow</td>
            <td className="py-2">Responds instantly</td>
          </tr>
          <tr>
            <td className="py-2">Privacy concerns</td>
            <td className="py-2">Your data stays private</td>
          </tr>
        </tbody>
      </table>

      <p className="text-sm text-gray-500 mt-4">
        Self-driving cars, smartwatches, and drones all use Edge AI!
      </p>
    </div>
  )}
 </div>
);
}
```

---

7. M3: PBL + Showcase

7.1 Project Brief Page

```typescript
```

```
// File: pages/ProjectBriefPage.tsx

import { useState } from 'react';
import { useSessionStore } from '../state/sessionStore';
import { useNavigate } from 'react-router-dom';

const USE_CASES = [
  { value: 'accessibility', label: 'Accessibility Helper' },
  { value: 'gaming', label: 'Gaming Controller' },
  { value: 'art', label: 'Art & Music' },
  { value: 'communication', label: 'Communication' },
  { value: 'other', label: 'Other' },
] as const;

export function ProjectBriefPage() {
  const { session, setProjectBrief } = useSessionStore();
  const navigate = useNavigate();

  const [studentName, setStudentName] = useState(session?.projectBrief?.studentName || '');
  const [projectName, setProjectName] = useState(session?.projectBrief?.name || '');
  const [problem, setProblem] = useState(session?.projectBrief?.problemStatement || '');
  const [useCase, setUseCase] = useState(session?.projectBrief?.useCase || 'other');
  const [gestureIdeas, setGestureIdeas] = useState(session?.projectBrief?.gestureIdeas || '');

  const canContinue = studentName.trim() && projectName.trim() && problem.trim();

  const handleContinue = () => {
    setProjectBrief({
      studentName: studentName.trim(),
      name: projectName.trim(),
      problemStatement: problem.trim(),
      useCase,
      gestureIdeas: gestureIdeas.trim(),
    });
    navigate('/preview');
  };

  return (
    <div className="max-w-xl mx-auto p-6">
      <h1 className="text-2xl font-bold mb-2">📋 Your AI Project</h1>
      <p className="text-gray-600 mb-6">
        Before we start, let's define what you're building and why.
      </p>
```

```
<div className="space-y-4">
  <div>
    <label className="block font-medium mb-1">Your Name</label>
    <input
      type="text"
      value={studentName}
      onChange={(e) => setStudentName(e.target.value)}
      placeholder="Emma Johnson"
      maxLength={50}
      className="w-full p-3 border rounded-lg"
    />
  </div>

  <div>
    <label className="block font-medium mb-1">Project Name</label>
    <input
      type="text"
      value={projectName}
      onChange={(e) => setProjectName(e.target.value)}
      placeholder="Magic Wand Controller"
      maxLength={30}
      className="w-full p-3 border rounded-lg"
    />
  </div>

  <div>
    <label className="block font-medium mb-1">What problem will your AI solve?</label>
    <textarea
      value={problem}
      onChange={(e) => setProblem(e.target.value)}
      placeholder="Help my little brother play games without needing the keyboard"
      maxLength={200}
      rows={3}
      className="w-full p-3 border rounded-lg resize-none"
    />
    <p className="text-xs text-gray-400 text-right">{problem.length}/200</p>
  </div>

  <div>
    <label className="block font-medium mb-1">Category</label>
    <select
      value={useCase}
      onChange={(e) => setUseCase(e.target.value as typeof useCase)}
```

```
          className="w-full p-3 border rounded-lg"
        >
          {USE_CASES.map((uc) => (
            <option key={uc.value} value={uc.value}>
              {uc.label}
            </option>
          ))}
        </select>
      </div>

      <div>
        <label className="block font-medium mb-1">Gesture ideas (optional)</label>
        <input
          type="text"
          value={gestureIdeas}
          onChange={(e) => setGestureIdeas(e.target.value)}
          placeholder="cast spell, slash, shield"
          className="w-full p-3 border rounded-lg"
        />
        <p className="text-xs text-gray-400">Separate with commas</p>
      </div>
    </div>

    <button
      onClick={handleContinue}
      disabled={!canContinue}
      className="w-full mt-6 py-3 bg-blue-600 text-white rounded-lg font-medium disabled:opacity-50"
    >
      Continue →
    </button>
  </div>
);
}
```

7.2 Badge System

```typescript
```

```
// File: badges/badges.ts

export interface BadgeDefinition {
  id: BadgeId;
  icon: string;
  name: string;
  criteria: string;
}

export const BADGES: Record<BadgeId, BadgeDefinition> = {
  connected: {
    id: 'connected',
    icon: '🔌',
    name: 'Connected',
    criteria: 'Successfully paired with Arduino',
  },
  'sensor-explorer': {
    id: 'sensor-explorer',
    icon: '🔍',
    name: 'Sensor Explorer',
    criteria: 'Passed the sensor quiz',
  },
  designer: {
    id: 'designer',
    icon: '🎨',
    name: 'Designer',
    criteria: 'Passed the gesture design quiz',
  },
  'data-scientist': {
    id: 'data-scientist',
    icon: '📊',
    name: 'Data Scientist',
    criteria: 'Recorded 10+ samples per gesture',
  },
  'ai-trainer': {
    id: 'ai-trainer',
    icon: '🧠',
    name: 'AI Trainer',
    criteria: 'Achieved 80%+ training accuracy',
  },
  'edge-engineer': {
    id: 'edge-engineer',
    icon: '🚀',
```

```
    name: 'Edge Engineer',
    criteria: 'Successfully deployed model to Arduino',
  },
  'sharp-shooter': {
    id: 'sharp-shooter',
    icon: '🎯',
    name: 'Sharp Shooter',
    criteria: '10 correct predictions in a row',
  },
};
```

typescript

```
// File: components/BadgeToast.tsx

import { useEffect, useState } from 'react';
import { BADGES } from '../badges/badges';
import { BadgeId } from '../storage/schema';

interface Props {
  badgeId: BadgeId;
  onClose: () => void;
}

export function BadgeToast({ badgeId, onClose }: Props) {
  const badge = BADGES[badgeId];

  useEffect(() => {
    const timer = setTimeout(onClose, 4000);
    return () => clearTimeout(timer);
  }, [onClose]);

  return (
    <div className="fixed bottom-4 right-4 bg-white shadow-xl rounded-xl p-4 flex items-center gap-3 animate-slide-up">
      <div className="text-4xl">{badge.icon}</div>
      <div>
        <p className="font-bold text-lg">Badge Earned!</p>
        <p className="text-gray-600">{badge.name}</p>
      </div>
    </div>
  );
}
```

7.3 Self-Contained HTML Portfolio Download

```typescript
```

```
// File: services/portfolioExportService.ts

import { StudentSession } from '../storage/schema';
import { BADGES } from '../badges/badges';

export function downloadPortfolio(session: StudentSession): void {
  const html = generatePortfolioHTML(session);
  const blob = new Blob([html], { type: 'text/html;charset=utf-8' });
  const url = URL.createObjectURL(blob);

  const filename = sanitizeFilename(session.projectBrief?.name || 'My_AI_Portfolio');
  const date = new Date().toISOString().split('T')[0];

  const a = document.createElement('a');
  a.href = url;
  a.download = `${filename}_${date}.html`;
  document.body.appendChild(a);
  a.click();
  document.body.removeChild(a);
  URL.revokeObjectURL(url);
}

function generatePortfolioHTML(session: StudentSession): string {
  const {
    projectBrief,
    gestures,
    badges,
    designJournal,
    trainingAccuracy,
    createdAt,
  } = session;

  const studentName = projectBrief?.studentName || 'A Severn Student';
  const projectName = projectBrief?.name || 'My Gesture AI';
  const problemStatement = projectBrief?.problemStatement || '';

  const formattedDate = new Date(createdAt).toLocaleDateString('en-US', {
    year: 'numeric',
    month: 'long',
    day: 'numeric',
  });

  const gestureCards = gestures
```

```
    .map(
      (g) => `
      <div class="gesture-card">
        <div class="gesture-icon">${getGestureEmoji(g.name)}</div>
        <div class="gesture-name">${escapeHtml(g.name)}</div>
      </div>
    `
    )
    .join('');

  const badgeList = badges
    .map((id) => {
      const badge = BADGES[id];
      return `<span class="badge">${badge.icon} ${badge.name}</span>`;
    })
    .join('');

  const reflections =
    designJournal.length > 0
      ? designJournal.map((e) => `<blockquote>"${escapeHtml(e.response)}"</blockquote>`).join('')
      : '';

  const accuracySection =
    trainingAccuracy !== null
      ? `<p class="accuracy">Training Accuracy: <strong>${Math.round(trainingAccuracy * 100)}%</strong></p>`
      : '';

  return `<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>${escapeHtml(projectName)} - AI Portfolio</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
      background: #f5f5f5;
      color: #333;
      line-height: 1.6;
      padding: 20px;
    }
    .certificate {
      max-width: 800px;
```

```
  margin: 0 auto;
  background: white;
  border-radius: 12px;
  box-shadow: 0 4px 20px rgba(0,0,0,0.1);
  padding: 40px;
}
header { text-align: center; margin-bottom: 20px; }
.logo { font-size: 48px; margin-bottom: 10px; }
h1 { font-size: 28px; letter-spacing: 4px; color: #2563eb; }
.subtitle { font-size: 16px; color: #666; font-style: italic; }
.divider {
  height: 1px;
  background: linear-gradient(to right, transparent, #ddd, transparent);
  margin: 25px 0;
}
.hero { text-align: center; padding: 20px 0; }
.student-name { font-size: 32px; font-weight: 600; color: #1e40af; }
.achievement { font-size: 18px; color: #555; }
h3 {
  font-size: 12px;
  letter-spacing: 2px;
  color: #888;
  margin-bottom: 15px;
  text-transform: uppercase;
}
.project-name { font-size: 24px; font-weight: 600; }
.problem-statement {
  font-style: italic;
  color: #555;
  padding-left: 20px;
  border-left: 3px solid #2563eb;
}
.gesture-grid {
  display: flex;
  flex-wrap: wrap;
  gap: 15px;
  justify-content: center;
  margin-bottom: 15px;
}
.gesture-card {
  background: #f8fafc;
  border: 2px solid #e2e8f0;
  border-radius: 12px;
  padding: 15px 25px;
```

```
  text-align: center;
}
.gesture-icon { font-size: 28px; margin-bottom: 5px; }
.gesture-name { font-size: 14px; font-weight: 600; color: #334155; }
.accuracy { text-align: center; }
.accuracy strong { color: #059669; font-size: 20px; }
.badge-grid {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  justify-content: center;
}
.badge {
  background: #fef3c7;
  border: 1px solid #fbbf24;
  border-radius: 20px;
  padding: 8px 16px;
  font-size: 14px;
}
blockquote {
  background: #f8fafc;
  border-left: 4px solid #2563eb;
  padding: 15px 20px;
  margin: 10px 0;
  font-style: italic;
  color: #475569;
  border-radius: 0 8px 8px 0;
}
.explainer {
  background: #eff6ff;
  border-radius: 8px;
  padding: 20px;
}
.explainer p { font-size: 14px; color: #1e40af; }
footer { text-align: center; padding-top: 10px; }
.date { font-size: 16px; font-weight: 600; }
.school { font-size: 14px; color: #666; }
.print-button { text-align: center; margin-top: 20px; }
.print-button button {
  background: #2563eb;
  color: white;
  border: none;
  padding: 12px 24px;
  border-radius: 8px;
```

```
    font-size: 16px;
    cursor: pointer;
  }
  @media print {
    body { background: white; padding: 0; }
    .certificate { box-shadow: none; }
    .print-button { display: none; }
  }
  @media (max-width: 600px) {
    .certificate { padding: 20px; }
    h1 { font-size: 22px; }
    .student-name { font-size: 24px; }
  }
 </style>
</head>
<body>
 <div class="certificate">
  <header>
    <div class="logo">🧠 </div>
    <h1>SEVERN EDGE AI</h1>
    <p class="subtitle">Certificate of Achievement</p>
  </header>

  <div class="divider"></div>

  <section class="hero">
    <h2 class="student-name">${escapeHtml(studentName)}</h2>
    <p class="achievement">trained an artificial intelligence<br>to recognize gestures</p>
  </section>

  <div class="divider"></div>

  <section>
    <h3>PROJECT</h3>
    <p class="project-name">${escapeHtml(projectName)}</p>
    ${problemStatement ? `
    <h4 style="font-size:11px;letter-spacing:1px;color:#888;margin:20px 0 10px;text-transform:uppercase;">PROBLEM SO
    <p class="problem-statement">"${escapeHtml(problemStatement)}"</p>
    ` : ''}
  </section>

  <div class="divider"></div>

  <section>
```

```
    <h3>GESTURES CREATED</h3>
    <div class="gesture-grid">${gestureCards}</div>
    ${accuracySection}
  </section>

  <div class="divider"></div>

  <section>
    <h3>BADGES EARNED</h3>
    <div class="badge-grid">
      ${badgeList || '<p style="color:#888;font-style:italic;">Keep going to earn badges!</p>'}
    </div>
  </section>

  ${designJournal.length > 0 ? `
  <div class="divider"></div>
  <section>
    <h3>REFLECTIONS</h3>
    ${reflections}
  </section>
  ` : ''}

  <div class="divider"></div>

  <section class="explainer">
    <h3>WHAT IS EDGE AI?</h3>
    <p>
      Your child trained a <strong>neural network</strong> — the same technology
      used in self-driving cars and smart watches. Unlike most AI that runs on
      big computers in the cloud, this AI runs entirely on a tiny chip the size
      of a postage stamp. That's called <strong>"Edge AI"</strong> — AI that runs
      on the edge, right where the action happens, with no internet needed.
    </p>
  </section>

  <div class="divider"></div>

  <footer>
    <p class="date">${formattedDate}</p>
    <p class="school">Severn School · 5th Grade</p>
  </footer>
</div>

<div class="print-button">
```

```
    <button onclick="window.print()">🖨️ Print This Page</button>
  </div>
</body>
</html>`;
}

function getGestureEmoji(name: string): string {
  const emojiMap: Record<string, string> = {
    wave: '👋', shake: '🤝', circle: '⭕', spell: '✨', magic: '🪄',
    slash: '🗡️', sword: '⚔️', shield: '🛡️', punch: '👊', throw: '🎯',
    spin: '🌀', flick: '👆', swipe: '👉', clap: '👏', point: '👆',
  };
  const normalized = name.toLowerCase().trim();
  for (const [key, emoji] of Object.entries(emojiMap)) {
    if (normalized.includes(key)) return emoji;
  }
  return '🎯';
}

function escapeHtml(text: string): string {
  const div = document.createElement('div');
  div.textContent = text;
  return div.innerHTML;
}

function sanitizeFilename(name: string): string {
  return name.replace(/[^a-z0-9]/gi, '_').replace(/_+/g, '_').substring(0, 50);
}
```

typescript

```
// File: components/ShareSection.tsx

import { useSessionStore } from '../state/sessionStore';
import { downloadPortfolio } from '../services/portfolioExportService';

export function ShareSection() {
  const { session } = useSessionStore();

  if (!session) return null;

  const handleDownload = () => {
    downloadPortfolio(session);
  };

  const handleEmail = () => {
    const subject = encodeURIComponent(
      `Look what I built! - ${session.projectBrief?.name || 'My AI Project'}`
    );
    const body = encodeURIComponent(`
Hi!

Today I trained my own AI to recognize gestures!

My project: ${session.projectBrief?.name || 'Gesture Recognition AI'}
${session.projectBrief?.problemStatement ? `Problem I solved: ${session.projectBrief.problemStatement}` : ''}

My gestures: ${session.gestures.map((g) => g.name).join(', ')}

What I learned:
${session.designJournal[0]?.response || 'I learned how AI sees patterns in numbers!'}

- Your kid 🎉

P.S. I also have a portfolio file to show you!
    `.trim());

    window.open(`mailto:?subject=${subject}&body=${body}`);
  };

  const isDisabled = session.gestures.length === 0;

  return (
    <div className="bg-white rounded-xl p-6 shadow-sm border">
```

```
    <h3 className="text-lg font-semibold mb-4">📤 Share Your Work</h3>

    <div className="space-y-4">
      <div>
        <button
          onClick={handleDownload}
          disabled={isDisabled}
          className={`flex items-center gap-2 px-4 py-2 rounded-lg font-medium ${
            isDisabled
              ? 'bg-gray-200 text-gray-400 cursor-not-allowed'
              : 'bg-blue-600 text-white hover:bg-blue-700'
          }`}
        >
          💾 Download Portfolio
        </button>
        <p className="text-sm text-gray-500 mt-2">
          Download an HTML file you can save to Google Drive and share.
        </p>
      </div>

      <div className="border-t pt-4">
        <button
          onClick={handleEmail}
          className="flex items-center gap-2 px-4 py-2 rounded-lg font-medium border border-gray-300 hover:bg-gray-50"
        >
          📧 Email to Family
        </button>
      </div>
    </div>

    <div className="mt-6 p-4 bg-blue-50 rounded-lg">
      <h4 className="font-medium text-blue-900 mb-2">💡 How to share with parents:</h4>
      <ol className="text-sm text-blue-800 space-y-1 list-decimal list-inside">
        <li>Click "Download Portfolio"</li>
        <li>Open Google Drive (drive.google.com)</li>
        <li>Drag the file into Drive</li>
        <li>Right-click → Share → Copy link</li>
        <li>Send the link to your family!</li>
      </ol>
    </div>
  </div>
);
}
```

8. Technical Reference

BLE Connection (The Post Office Metaphor)

```
Arduino = Post Office
├── GATT Server = Building is open
├── Service = A section (Sensors, Models)
├── Characteristic = A specific PO box
└── Notify = "Call me when mail arrives"

Connection sequence:
1. requestDevice()      → Find the building
2. gatt.connect()       → Open the front door
3. getPrimaryService()  → Walk to the Sensor Section
4. getCharacteristic()  → Find the specific PO box
5. startNotifications() → "Call me when new mail arrives"

On disconnect:
- You're kicked outside
- You still know the address
- You MUST redo steps 2-5 (full rehydration)
```

File Structure After Implementation

```
web-app/src/
├── App.tsx
├── pages/
│   ├── ConnectPage.tsx
│   ├── ProjectBriefPage.tsx       # NEW (M3)
│   ├── PreviewPage.tsx            # ADD: Gate 1 (M2)
│   ├── CollectPage.tsx            # ADD: Gate 2 (M2)
│   ├── TrainPage.tsx              # ADD: IdleClassBanner (M2)
│   ├── TestPage.tsx               # ADD: EdgeAIFactsPanel, ShareSection (M2/M3)
│   └── PortfolioPage.tsx          # NEW (M3)
├── components/
│   ├── ConnectionStatusPill.tsx   # NEW (M1)
│   ├── ReconnectModal.tsx         # NEW (M1)
│   ├── SessionRecoveryBanner.tsx  # NEW (M1)
│   ├── KnowledgeCheckModal.tsx    # NEW (M2)
│   ├── IdleClassBanner.tsx        # NEW (M2)
```

```
│   ├── EdgeAIFactsPanel.tsx       # NEW (M2)
│   ├── BadgeTray.tsx              # NEW (M3)
│   ├── BadgeToast.tsx             # NEW (M3)
│   ├── JournalPrompt.tsx          # NEW (M3)
│   ├── ShareSection.tsx           # NEW (M3)
│   └── DownloadPortfolioButton.tsx # NEW (M3)
├── state/
│   ├── connectionStore.ts         # NEW (M1)
│   └── sessionStore.ts            # NEW (M1)
├── storage/
│   ├── db.ts                      # NEW (M1)
│   └── schema.ts                  # NEW (M1)
├── services/
│   ├── bleService.ts              # MODIFY: Use connection store (M1)
│   ├── bleModelUploadService.ts   # MODIFY: Add retry (M1)
│   └── portfolioExportService.ts  # NEW (M3)
├── badges/
│   └── badges.ts                  # NEW (M3)
├── data/
│   └── knowledgeChecks.ts         # NEW (M2)
└── utils/
    └── debounce.ts                # NEW (M1)
```

---

9. Testing Requirements

Manual Test Scenarios

M1: Connection & Persistence

```
1. Connect to Arduino
2. Record 5 samples
3. Refresh page (Cmd+R)
4. Verify "Found Your Previous Work" banner appears
5. Click Continue
6. Verify 5 samples still exist

7. Connect to Arduino
8. Unplug Arduino USB
9. Verify "Reconnecting..." modal appears
10. Plug Arduino back in
11. Verify auto-reconnect succeeds
```

12. Connect to Arduino
13. Unplug Arduino USB
14. Wait for all retries to fail
15. Verify "Couldn't Reconnect" modal appears
16. Click "Choose Device"
17. Verify Bluetooth picker opens

M2: Knowledge Checks

1. Complete Connect
2. Click Next to go to Preview
3. Answer sensor quiz incorrectly
4. Verify shake animation + retry option
5. Answer correctly
6. Verify badge toast appears
7. Verify can proceed to Collect
8. Press Ctrl+Shift+U
9. Verify unlock button appears

M3: Portfolio Download

1. Complete full flow with:
   - Project Brief filled
   - 3 gestures trained
   - Journal entries written
2. Click "Download Portfolio"
3. Open downloaded .html file
4. Verify all sections render
5. Click "Print This Page"
6. Verify print preview looks good
7. Open file on phone
8. Verify mobile responsive

Unit Tests (Jest)

```typescript
```

```typescript
// state/connectionStore.test.ts
describe('ConnectionStore', () => {
  it('transitions from disconnected to connecting on startConnecting', () => {
    // ...
  });

  it('increments reconnectAttempt on startReconnecting', () => {
    // ...
  });

  it('resets to disconnected after 3 failed reconnects', () => {
    // ...
  });
});

// storage/db.test.ts (with fake-indexeddb)
describe('SessionDB', () => {
  it('saves and retrieves session', async () => {
    // ...
  });

  it('returns most recent session by lastUpdated', async () => {
    // ...
  });
});
```

Summary

This specification covers everything needed to implement the Severn Edge AI roadmap:

Milestone	Key Deliverables
M1	Connection state machine, IndexedDB persistence, session recovery, BLE upload retry
M2	2 knowledge check gates, "Idle" class explanation, Edge AI facts panel
M3	Project Brief, Design Journal, Badge system, Portfolio page, HTML download for parents

All code examples are production-ready TypeScript/React. Follow the dependency graph and implement in order.

End of specification.